



Modo IA

Tudo

Imagens

Vídeos

Notícias

Mais ▾



E aí! Olha, eu uso o Claude Code pra refatorar e migrar código, mas é exaustivo: ele faz um pedaço, trava e par... ▾

O medo do loop infinito é super justo! Deixar um agente de IA rodando sozinho no terminal dá aquele frio na barriga de acordar com uma fatura astronômica. [🔗](#)

Mas a boa notícia é que o comando `/goal` foi projetado com mecanismos de segurança bem específicos para evitar que isso aconteça. [🔗](#) Prospere AI +1

Aqui está uma explicação simples de como o `/goal` funciona nos bastidores e como você pode se proteger para dormir em paz. [🔗](#) Reddit · r/ClaudeAI

Como o `/goal` sabe que realmente terminou?

O Claude Code não usa apenas o bom senso para decidir se acabou; ele usa uma arquitetura de dupla validação: [🔗](#) Medium · Vasu Yadav +1

1. **O Executor (Claude Principal):** Ele planeja, mexe nos arquivos, roda comandos no terminal e tenta resolver o problema.
2. **O Avaliador (Subagente):** Toda vez que o Claude principal termina uma rodada (um "turno"), entra em ação um segundo modelo em segundo plano (geralmente o Haiku, que é super rápido e barato). Esse subagente lê o histórico e avalia friamente: *"A condição que o usuário pediu foi atingida de forma verificável?"* Se a resposta for não, ele dá o feedback do que falta e manda o executor continuar. Ele só para quando o objetivo é atingido de forma concreta. [🔗](#) YouTube · Felipe Rocha... +4

Pergunte o que quiser



Modo IA Tudo Imagens Vídeos Notícias Mais ▾



- **Limite de "Check-ins" Ociosos:** Enquanto o trabalho está rodando, o Claude faz verificações de status. Atualmente, o sistema limita essas verificações automáticas quando a sessão está ociosa (máximo de 3 check-ins) para evitar consumo fantasma de tokens. Medium · Automation Labs
- **Erros Críticos:** Se o terminal retornar um erro de sistema ou algo que exija intervenção humana real, o Claude limpa o objetivo e para imediatamente para não quebrar nada. code.claude.com

O segredo: Como escrever o `/goal` com segurança

Para garantir proteção total, **nunca** dê ordens vagas como `/goal refatore o projeto`. Em vez disso, coloque travas de segurança diretamente no comando. MindStudio +1

1. Sempre dê um critério de saída numérico ou temporal code.claude.com

Adicione um limite de turnos no fim do seu comando. Se ele chegar nesse limite e não terminar, ele para de qualquer forma. code.claude.com +1

Exemplo: `/goal Migre o módulo de autenticação para a nova API até que 'npm run build' passe sem erros, OU pare após 15 turnos.` code.claude.com

2. Dê uma linha de chegada baseada em testes ou compilação YouTube · Felipe Rocha ... +1

A IA precisa de um veredito binário do terminal (passou ou falhou) para ter certeza de que terminou. code.claude.com



Modo IA Tudo Imagens Vídeos Notícias Mais ▾



`user.service.ts` para usar Prisma. Restrição: Não altere nenhum arquivo dentro da pasta `/config` ou `/routes`. code.claude.com



Dicas de ouro antes de ir dormir:

- **Sempre use Git:** Faça um `git commit` ou `git stash` de tudo antes de disparar o comando. Se a IA fizer besteira enquanto você dorme, um simples `git reset --hard` resolve o seu problema. Hora de Codar
- **De olho nos custos:** De vez em quando, use o comando `/usage` ou `/cost` para ver o quanto a sua sessão atual já consumiu e configurar limites na sua conta da Anthropic. code.claude.com +2

Se quiser, compartilhe comigo:

- Qual é a **tarefa exata** (ex: migrar de uma biblioteca X para Y, quebrar um arquivo grande) que você precisa fazer?
- Qual **comando do seu terminal** (como um linter, teste ou build) diz se o código está funcionando?

Posso te ajudar a **escrever o comando** `/goal` **perfeito e blindado contra loops** para você testar hoje mesmo!



code.claude.com

Keep Claude working toward a goal - Claude Code Docs

13 de set. de 2026 — Agents and parallel work * Overview. * Create custom subagents. * Agent view. * Run agent teams. * Cross-session messaging. *...





Modo IA Tudo Imagens Vídeos Notícias Mais ▾



- O que você precisa **migrar ou refatorar** no seu código agora?
- Qual é o **comando de teste ou build** do seu projeto?

Estou pronto para montar o `/goal` perfeito para o seu caso!



Apresentar o Avaliador (Evaluator): Explicar que, a cada turno, um modelo rápido (como o Haiku) verifica se o ...



Exatamente!

É exatamente esse o mecanismo.

O **Avaliador (Evaluator)** funciona como um "gerente de controle de qualidade" implacável que roda em segundo plano. O fluxo do loop autônomo funciona assim a cada ciclo:

1. **O Executor age:** O modelo principal mexe no código, roda um comando e faz uma alteração.
2. **O Avaliador entra em cena:** Um modelo rápido e focado (como o Haiku) assume por um instante. Ele não altera arquivos; ele apenas lê o que foi feito e se pergunta: *"O objetivo final que o usuário digitou no `/goal` foi 100% atingido?"*
3. **A tomada de decisão:**
 - **Se NÃO foi cumprido:** O Avaliador assume o papel do seu "continua" automático. Ele aponta o que ainda está faltando e dispara o próximo turno para o Executor.



Modo IA Tudo Imagens Vídeos Notícias Mais ▾



Vender Autonomia com Responsabilidade: Demonstrar
que isso destrava trabalho de fundo (de minutos a hora... ▾

O grande trunfo do `/goal` é exatamente este: **destravar o trabalho de fundo (background work) com responsabilidade.**

Ao usar esse comando, você deixa de ser um "digitador de 'continua'" e passa a atuar como um **diretor de engenharia**. Você delega tarefas complexas que podem levar de **vários minutos a algumas horas** para o Claude Code resolver sozinho enquanto você foca em outra coisa ou vai descansar.

No entanto, autonomia no terminal exige uma contrapartida clara do desenvolvedor:

1. O Critério de Pronto (Definition of Done) precisa ser cirúrgico

Como o Avaliador (Haiku) é literal, se você der um objetivo vago, ele pode manter o loop rodando infinitamente tentando adivinhar o que você quer. O seu critério de sucesso precisa ser binário e incontestável para a máquina.

2. Atenção vermelha ao consumo de tokens

Sessões longas e autônomas consomem **muitos tokens**. Cada turno acumula o histórico dos turnos anteriores (contexto inflado). Se o Claude Code precisar de 20 ou 30 turnos para resolver uma migração complexa, o custo daquela única sessão pode subir rapidamente.



Modo IA Tudo Imagens Vídeos Notícias Mais ▾



- Qual **comando de validação** (como `npm run build` ou `pytest`) garante que ele não vai quebrar o sistema?

Com isso, montamos um `/goal` autônomo, mas totalmente responsável e controlado.

A IA pode cometer erros. Por isso, cheque as respostas

